

ASET Seed 0.1-rc11

Цифровое конституционное ядро для федеративных пространств доверия

Document ID: ASET-SEED-SPEC-001

Version: 0.1-rc11

Date: 2026-08-04

Status: DOCUMENTATION_FREEZE_READY

Predecessor: ASET Seed 0.1-rc10 draft; last complete release baseline 0.1-rc9

Reference profile: REFERENCE_STATE_MACHINE_CANDIDATE

Independent black-box re-audit: PASS_WITH_LIMITATIONS – exact-byte clean-room release audit

External third-party audit: PENDING

Implementation conformance: NOT_EXECUTED_PRE_IMPLEMENTATION

Production conformance: HOLD

0. Управление документом

Этот пакет определяет нормативную модель ASET Seed, типизированные JSON Schemas, исполняемую reference state machine, trace-based conformance corpus и отдельный adversarial re-audit harness.

Пакет не утверждает:

- юридическую действительность конкретной Trust Constitution;
- абсолютную истинность утверждений о физическом мире;
- корректность ещё не созданной production-реализации;
- наличие распределённого consensus-протокола;
- завершение внешнего аудита независимой третьей стороной.

Слова **MUST**, **MUST NOT**, **SHOULD**, **MAY** имеют нормативный смысл. Русский текст является нормативным. Машиночитаемые JSON Schemas и reference state machine являются исполняемыми носителями admission semantics. При расхождении между прозой и машинным профилем релиз считается внутренне несогласованным и не может получить production status.

1. Каноническая идея

ASET Seed - минимальное машиноисполняемое конституционное ядро, которое преобразует правила федерации, доказуемые полномочия, предъявленные результаты и evidence в канонические переходы состояния локальных нормативных namespaces.

```
ASET Seed semantics
+ authenticated Root Genesis
+ Trust Constitution
= one stateful Trust Space
```

Seed не является организацией, сувереном, моральным арбитром или автономным агентом. Trust Constitution не является Principal и не обладает собственной волей. Она является каноническим описанием:

- namespace-структуры;
- нормативных позиций;
- Capability и Authority;
- процедур **counts-as**;
- правил предъявления, Verification и Outcome;
- неизменяемых границ Genesis и правил переопределения membership Context;
- границ ответственности;
- dependency graph, необходимого для локализации последствий.

Seed действует как reference monitor: он не решает, справедлива ли Конституция, а проверяет, был ли кандидат перехода допустим по действующей Конституции и текущему состоянию.

2. Три уровня конструкции

2.1 ASET Seed

Универсальная семантика переходов, canonical encoding, hashing, state invariants и fail-closed admission.

2.2 Trust Constitution

Конкретная политика одного Trust Space: правила, immunities, capability requirements, coordination classifications, bootstrap policy и нормативные зависимости.

2.3 Trust Space

Конкретный stateful объект одной Genesis lineage. Он содержит неизменяемую идентичность, дерево namespaces, Authority bindings, Decisions, Permits, receipts, Observations, Verifications, Outcomes, exports/imports, membership withdrawals, context redefinitions, corrections и transition records.

Один runtime MAY обслуживать несколько Trust Spaces, но MUST изолировать их идентичности, состояния, ключи и histories.

3. Семь концептуальных примитивов

Seed содержит ровно семь примитивов:

1. **Authority** - доказуемая активная institutional **POWER** в точном Context и state.
2. **Decision** - нормативный выбор или readiness declaration, зафиксированный субъектом.
3. **Permit** - ограниченное разрешение на предъявление результата конкретного действия.
4. **ExecutionIntent** - конкретное использование Permit, материализованное при принятом предъявлении.
5. **Observation** - утверждение о результате или внешнем факте.
6. **Verification** - признание того, что утверждение прошло предписанную процедуру проверки.
7. **Outcome** - финальное институциональное признание результата действия.

Genesis, Constitution, Context, AuthorityBinding, PermitUseReceipt, ExportReceipt, ReconciliationReceipt, MembershipWithdrawalRecord, ContextRedefinitionRecord и CorrectionRecord являются обязательными протокольными объектами, но не дополнительными примитивами.

4. Упрощённое формальное ядро

Семь примитивов не требуют семи отдельных логик. Reference profile использует четыре отношения:

```
holds(holder, position, counterparty, object, context, state)
counts_as(event, transition, constitution, context)
supports(evidence, claim, policy, context)
depends_on(dependent, predecessor, kind)
```

Hohfeldian positions:

```
CLAIM      <-> DUTY
PRIVILEGE  <-> NO_CLAIM
POWER      <-> LIABILITY
IMMUNITY   <-> DISABILITY
```

Authority является активной **POWER**, а constitutional invariants выражаются как **IMMUNITY**, которой соответствует **DISABILITY** неуполномоченных субъектов.

Физическая возможность вызвать API, владение файлом или сетевой доступ не создают Authority.

5. Context как namespace

Context является нормативным namespace канонического состояния. Контекст субъекта существует внутри контекста федерации:

```
/
├─ federation F
│   └─ subject H
│       └─ organization O
│           └─ federation G
│               └─ subject K
```

5.1 Идентичность и адрес

```
ContextID - неизменяемая идентичность Genesis lineage;
ContextPath - локально разрешаемый адрес;
Alias - человекочитаемое имя.
```

Нормативные ссылки используют **ContextID**. Alias MAY быть переиспользован после окончательного прекращения ветви, поскольку live alias index содержит только **ACTIVE** Context. Историческая ссылка всегда использует неизменяемый **ContextID**; новый Context получает новый **ContextID** даже при том же alias.

Reference formula:

```
ContextID = H_domain(
  parent_context_id,
  context_genesis_digest
)
```

Смена родительского context не является **move**: она требует нового **MemberContextGenesis**.

5.2 Вложенность не создаёт Authority

```
Namespace ancestry != Authority inheritance
```

Родитель не получает автоматического права изменять внутреннее состояние ребёнка. Ребёнок не может действовать от имени родителя. Любая Authority должна быть явно создана и доказана.

5.3 Одно действие - один Context

Каждое действие принадлежит ровно одному **context_id**. Cross-context process является цепочкой локальных действий, ExportReceipt, Import Observation и локального Outcome, а не одним действием в нескольких namespaces.

6. Root Genesis и bootstrap

6.1 Пустой корень

`initialize_state(root_genesis)` создаёт ровно один root Context:

- без членов;
- без Authority;
- без Permits;
- без Outcomes;
- без дочерних namespaces.

Root Constitution присутствует, но root Context не содержит скрытого владельца.

6.2 Precommitted bootstrap

Первое членство создаётся не Authority внутри пустого корня, а внешне закреплённым bootstrap predicate, входящим в Root Genesis:

```
validator_principal_id
max_admissions
allowed_context_kinds
allowed_initial_capabilities
```

Первый кандидат не может установить правило собственного принятия. После исчерпания `max_admissions` bootstrap необратимо закрывается.

6.3 Идентичность Trust Space

```
constitution_digest = H_domain(canonical Constitution)
root_genesis_digest = H_domain(genesis material)
trust_space_id = H_domain(
    seed_semantics_id,
    root_genesis_digest,
    external_anchor_digest
)
```

Genesis сам не назначает себе доверие. Заявленные digests и IDs должны совпадать с внешне закреплёнными значениями.

7. Canonical encoding и hashing

Reference profile требует:

- UTF-8 JSON;
- rejection duplicate members на parse boundary;
- Unicode NFC;
- sorted object keys;
- integer-only numeric profile;
- отсутствие float и NaN;
- SHA-256;
- domain separation;
- length framing.

```
digest = SHA256(
    ASCII(domain) || 0x00 || uint64_be(length) || canonical_bytes
)
```

Self-certifying IDs используют отдельные domains для TrustSpaceID, ContextID, TransitionID, ArtifactID, AuthorityBinding, LocalCommit и StateRoot.

8. Transition envelope

Каждый обычный переход имеет единый envelope:

```

schema_version
transition_id
trust_space_id
context_id
kind
parent_state_root
expected_local_ordinal
constitution_epoch
causal_parents[]
authn {
  signer_principal_id
  proof_digest
}
payload

```

`transition.schema.json` является discriminated union: каждый `kind` имеет отдельную strict payload schema. Неизвестные поля запрещены. Нормативные schemas применяются не только conformance runner, но и непосредственно публичными функциями `initialize_state`, `apply_transition` и `validate_state`. Schema-invalid или malformed input MUST завершаться стабильным fail-closed кодом без частичного изменения State и без необработанного исключения.

Кандидат предоставляет первичные данные и proof references. Он не может передавать derived booleans вроде `scope_subset`, `fork_detected`, `first_invalid_index`, `affected_contexts` или `negative_basis_complete`. Эти свойства вычисляет Seed.

9. Нормативный интерфейс state machine

```

initialize_state(RootGenesis) -> State0

apply_transition(StateN, Transition)
-> ACCEPTED(StateN+1, artifacts)
| REJECTED(code, unchanged StateN)
| IDEMPOTENT_REPLAY(unchanged StateN)
| IDEMPOTENT_SUBMISSION_REPLAY(unchanged StateN)

compute_state_root(State) -> Digest
validate_state(State) -> PASS | error

```

Принятый переход атомарно:

1. проверяет envelope и Authority;
2. вычисляет derived predicates;
3. материализует protocol artifacts;
4. обновляет state;
5. проверяет whole-state invariants;
6. вычисляет новый state root.

При любой ошибке исходное состояние возвращается без изменения.

`causal_parents` не являются произвольным заявлением кандидата. Seed извлекает `causal_basis_refs` из типизированных ссылок payload, разрешает каждый artifact в transition, который его создал, вычисляет обязательное множество причинных родителей и требует точного совпадения. Transition record сохраняет и `artifact_refs`, и `causal_basis_refs`; whole-state validation повторно вычисляет их связь.

`accepted_transition_count` является техническим индексом сериализации reference implementation. Normative causality задаётся вычисленным DAG; индекс не превращает все действия в одну нормативную причинную линию.

10. Authority

AuthorityBinding содержит:

```
context_id
capability_kind
scope
scope_digest
holder_principal_id
authority_epoch
status
grant_provenance
```

AuthorityKey:

```
ContextID :: CapabilityKind :: ScopeDigest
```

Для одного AuthorityKey в принятом состоянии допускается не более одного активного holder независимо от epoch. Кроме того, для одной пары (**ContextID**, **CapabilityKind**) active scopes MUST быть попарно непересекающимися. Wildcard * пересекается с любым scope; для конечного reference profile пересечение остальных scopes определяется непустым пересечением их нормализованных множеств. Epoch является provenance версии Authority, а не частью exclusivity key.

Static role, alias или self-assertion не создают Authority.

10.1 Передача Authority

Передача Authority требует положительного Outcome действия, task digest которого точно описывает передачу конкретного AuthorityBinding новому holder. Новый holder MUST заранее подписать **READINESS_ACCEPT_RESPONSIBILITY**, связанную с передаваемым AuthorityBinding и exact transfer terms. Он же MUST быть delegate transfer Permit. Исполнитель, старый holder или третья сторона не могут принять ответственность за него.

Вся action lineage передачи — readiness, **ISSUE_PERMIT** Decision, Permit, **ExecutionIntent**, **PermitUseReceipt**, Observation, Verification и Outcome — MUST принадлежать тому же Context, что и передаваемый AuthorityBinding. Outcome дочернего или соседнего namespace не может изменить Authority другого Context. Межконтекстное основание сначала проходит Export/Import и затем отдельное локальное признание в Context Authority.

Принятый **AUTHORITY_TRANSFER** атомарно:

- закрывает старый interval статусом **TRANSFERRED**;
- создаёт новый binding с **epoch + 1**;
- не переписывает историческую ответственность;
- не переносит открытые Permit автоматически.

11. Decision и readiness

Readiness является **Decision**, а не отдельным примитивом.

```
READINESS_EXECUTE
READINESS_ACCEPT_RESPONSIBILITY
```

Readiness подписывается самим субъектом и связывает:

- точный Context;
- subject principal;
- scope;
- conditions digest.

Она не активирует ответственность и не признаёт изменение состояния. Permit не может быть выдан без совместимой readiness и отдельного **ISSUE_PERMIT** Decision действующей Authority.

Reference profile вычисляет единый **PermitTermsDigest** из delegate, task, scope, success predicate, attempt limit, validity и caveats. И readiness, и **ISSUE_PERMIT** Decision MUST содержать этот exact digest; subject обоих Decisions MUST совпадать с delegate. Старые Decisions из предыдущей Constitution epoch не могут использоваться для нового Permit.

12. Permit и попытки

Permit определяет:

```

issuer
delegate
decision_ref
readiness_ref
task_digest
scope
success_predicate_digest
max_attempts
attempts_used
stop_on_positive = true
validity_end_ordinal
caveats

```

Permit MAY разрешать 1..N предъявлений. В минимальном профиле `stop_on_positive` фиксирован в `true`: признанный положительный Outcome всегда завершает действие.

12.1 Граница Seed

Подготовка результата находится вне Seed. Сюда относятся MCP calls, сеть, локальные tools, физическая работа и промежуточные вычисления.

Попытка расходуется только тогда, когда Seed принимает предъявление для Validation и атомарно создаёт:

```

ExecutionIntent
PermitUseReceipt

```

12.2 Idempotency

- exact replay одного TransitionID возвращает `IDEMPOTENT_REPLAY`;
- новая доставка с тем же `submission_id`, Permit и candidate digest возвращает `IDEMPOTENT_SUBMISSION_REPLAY`;
- тот же `submission_id` с иным digest является collision;
- успешный replay не увеличивает `attempts_used`.

12.3 Attenuation

Attenuation является линейной, а не копирующей операцией. Она:

```

закрывает parent Permit статусом ATTENUATED;
требует readiness нового delegate;
создаёт ровно один active child Permit;
child.scope subset-of parent.scope;
child.max_attempts <= parent.remaining_attempts;
child.validity <= parent.validity;
child.caveats include every parent caveat unchanged.

```

Неиспользованный остаток, не переданный ребёнку, прекращается; parent и child не могут одновременно расходовать один и тот же attempt budget. Эти свойства вычисляются Seed, а не передаются кандидатом как booleans.

13. Observation, Verification и Outcome

13.1 Observation

Observation является claim, связанным с конкретными:

- Permit;
- PermitUseReceipt;
- Context;
- claim digest;
- evidence references.

13.2 Verification

Verification связывает Observation с процедурой проверки. Она содержит verifier, policy digest, status и result class.

Seed гарантирует, что Verification:

- произведена уполномоченным verifier;
- использует признанную policy;
- связана с тем же Permit, receipt, Observation и Context;
- имеет допустимую пару `status/result_class`.

`policy_digest` MUST точно разрешаться в значение активной записи `Trust Constitution.rules` текущей Constitution epoch. Произвольный digest, отсутствующий в активном реестре правил, отклоняется с `VERIFICATION_POLICY_UNRECOGNIZED`. Наличие Authority verifier не позволяет создать новую процедуру проверки вне Конституции.

Качество сенсоров, людей, MCP и внешних источников является ответственностью федерации и реализации. Verification признаёт прохождение процедуры, но не доказывает абсолютную физическую истину.

13.3 Outcome

Outcome является финальным институциональным признанием результата Permit. Он принимает только `POSITIVE` или `NEGATIVE`.

Outcome вычисляется из полного effective набора PASS Verification данного Permit. Candidate не может выбрать удобное подмножество. `POSITIVE` требует effective SUCCESS; `NEGATIVE` требует verified FAILURE, отсутствие effective SUCCESS и terminal condition Permit. `FAIL` и `UNKNOWN` Verification не создают Outcome.

Внутри минимального Seed Outcome immutable: Correction не может иметь Outcome target. После финализации изменение институционального решения требует отдельного append-only review process или нового действия с собственной Authority, Permit и Outcome. Исторический Outcome не переписывается и не превращается обратно в открытое действие.

14. Межконтекстное взаимодействие

14.1 Export

Source Context предъявляет ожидаемый предыдущий `source_export_root`. Seed проверяет его и вычисляет новый export commitment из previous root, claim, optional local Outcome и TransitionID. ExportReceipt содержит:

- `source_context_id`;
- `previous_export_root`;
- новый вычисленный `source_export_root`;
- claim digest;
- source guarantee status;
- optional source Outcome, существующий в том же Context;
- `transition_ref`.

Обычный **EXPORT** разрешён только при **CONFIRMED** guarantee. При partition используются local commits и последующая reconciliation.

14.2 Import

Получатель не импортирует чужой Outcome как свой. **IMPORT** требует существующие локальные Permit и PermitUseReceipt, принадлежащие presenter в target Context, затем создаёт локальный ImportRecord и локальную Observation. Далее получатель выполняет собственную Verification и принимает собственный Outcome.

```
foreign Outcome -> evidence
not -> automatic local Outcome
```

15. Partition и федеративная гарантия

При сетевом разделении федерация приостанавливает гарантию только конкретной дочерней ветви:

```
Guarantee(F, child) = SUSPENDED
```

Идентичность субъекта и его независимые namespaces в других федерациях не изменяются.

Ранее подтверждённая история сохраняется. Внутренняя ветвь MAY продолжать только операции, предварительно классифицированные Конституцией:

```
MONOTONE_LOCAL
INVARIANT_CONFLUENT
COORDINATION_REQUIRED
```

Неизвестный operation class fail-closed классифицируется как **COORDINATION_REQUIRED**.

INVARIANT_CONFLUENT требует заранее признанного proof digest. Candidate не определяет собственный coordination class.

Статус **SUSPENDED** является общим gate до dispatch transition handler. Обычные **DECISION**, **PERMIT_ISSUE**, **OUTCOME**, Authority changes и иные transitions в таком Context запрещены. Единственный reference entry point локального продолжения — **PARTITION_LOCAL_TRANSITION**, который заново проверяет Constitution-defined coordination class и proof.

16. Reconciliation и fork consistency

Reconciliation начинается от `last_confirmed_export_root`.

Каждый local commit содержит:

```
commit_id
parent_export_root
new_export_root
operation_class
commit_digest
signer_principal_id
proof_digest
```

Seed независимо вычисляет `commit_id`, `new_export_root`, допустимость signer и coordination class. Если State уже содержит unconfirmed commits, reconciliation package **MUST** включать каждый известный commit; кандидат не может скрыть известную альтернативную ветвь. Дополнительные полученные commits допустимы только после полной проверки.

Результаты:

CONFIRMED
PARTIALLY_CONFIRMED
FORK_DETECTED
INSUFFICIENT_EVIDENCE

Первый недопустимый commit и его причинные потомки не подтверждаются. Допустимый префикс MAY быть принят. Два разных descendants одного parent root образуют fork и не могут быть молча объединены. При **FORK_DETECTED** известные competing commits сохраняются как persistent fork evidence; reconciliation не очищает их и не превращает одну выбранную ветвь в единственную известную историю.

17. Добровольный выход и атомарное переопределение Context

Root Constitution и root Context неизменяемы внутри одной Genesis lineage. Любое изменение их смысла требует нового Genesis. Seed не содержит общего языка patching Конституции, рекурсивного плебисцита, скрытого согласия молчанием или промежуточного **PENDING** governance-state.

Governance разделяет два действия с разной семантикой:

1. **MEMBERSHIP_WITHDRAW** — самостоятельный окончательный выход одного member Context;
2. **CONTEXT_REDEFINE** — одно атомарное parent-authorized преобразование точного множества нормативно связанных прямых siblings.

17.1 Самостоятельный **MEMBERSHIP_WITHDRAW**

Переход принадлежит отзываемому Context и **MUST** быть подписан его **member_principal_id**. Root Context отозвать нельзя. Payload содержит только **reason_digest**; выход не обещает successor и не является согласием на будущий proposal.

До изменения State Seed вычисляет **AffectedSiblingSet** относительно самого Context. Если другой активный прямой sibling нормативно зависит от него, автономный выход отклоняется с **WITHDRAWAL_REDEFINITION_REQUIRED**: сначала parent обязан атомарно переопределить зависимые siblings либо удалить зависимость отдельным допустимым действием будущего профиля. Это исключает активную ветвь, ссылающуюся на historical Context.

Принятый выход атомарно:

- переводит Context и descendants в **WITHDRAWN**;
- отзывает их active Authority;
- завершает их active Permit с **TERMINATED_WITH_CONTEXT**;
- удаляет aliases subtree из live alias index;
- сохраняет всю историю и неизменяемые ContextID;
- создаёт **MembershipWithdrawalRecord(mode = VOLUNTARY_EXIT)** с member principal, reason digest, authentication proof digest и точным withdrawn subtree.

Alias освобождается после commit выхода. Его повторное использование создаёт новый Genesis и новый **ContextID**; это не возобновляет прежнюю lineage.

17.2 **AffectedSiblingSet**

Для target — непосредственного child указанного parent — Seed вычисляет минимальное замкнутое множество прямых siblings:

- target входит всегда;
- если активный прямой sibling имеет **NORMATIVE** dependency на уже включённый Context, он также включается;
- правило применяется транзитивно до неподвижной точки;
- зависимости descendants сворачиваются к владельцу — непосредственному child данного parent;
- **EVIDENTIAL**, **INTERFACE**, **RESPONSIBILITY** и иные ненормативные edges не расширяют множество;
- Context другого parent, historical Context и неизвестный Context не могут быть включены.

Множество вычисляется из pre-state и не зависит от порядка authorizations. Candidate не передаёт derived boolean или готовый affected set как доверенное утверждение.

17.3 Canonical **ContextRedefinitionProposal**

Proposal является полностью материализованным объектом внутри payload **CONTEXT_REDEFINE**:

```
parent_context_id
target_context_id
proposal_nonce
replacements[] {
  old_context_id
  context_genesis_nonce
  initial_authorities[]
  depends_on_context_ids[]
}
```

proposal_digest вычисляется domain-separated hash от canonical proposal целиком. Он связывает parent, target, nonce, exact replacement set, authority bootstrap каждого successor и новый dependency graph. Оpaque digest без вложенного proposal недостаточен и schema-invalid.

17.4 Member authorizations

Тот же transition содержит exact `withdrawal_authorizations[]`. Для каждого Context из `AffectedSiblingSet` требуется ровно одна запись:

```
context_id
member_principal_id
proposal_digest
proof_digest
```

`member_principal_id` MUST совпадать с member старого Context, а `proposal_digest` — с canonical digest вложенного proposal. Отсутствующая, лишняя, повторная, подменённая или относящаяся к другому proposal authorization отклоняет весь transition до мутации State.

Authorization является evidence внутри атомарного перехода, а не отдельным destructive transition. Поэтому не возникает `PENDING` Context, временной потери Authority, alias reservation, cancellation protocol или nested-withdrawal race.

17.5 Parent Authority и atomic commit

`CONTEXT_REDEFINE` принадлежит parent Context и требует active Authority capability `REDEFINE_CONTEXT` в exact parent namespace. Signer transition MUST быть holder этой Authority. Proposal `parent_context_id` MUST совпадать с envelope `context_id`.

До первой мутации Seed MUST проверить:

- exact `AffectedSiblingSet`;
- exact authorization set и member signatures;
- proposal digest;
- uniqueness replacement old IDs;
- отсутствие successor ID collision;
- сохранение parent, local alias, member principal и context kind;
- допустимость initial Authority и отсутствие collision;
- существование и lifecycle всех dependency targets;
- отсутствие новой ссылки на descendant, который будет withdrawn этим же commit;
- exact remapping dependencies между одновременно заменяемыми siblings.

После успешной проверки один commit:

1. переводит старые affected Context в `SUPERSEDED`, а их descendants — в historical withdrawn subtree;
2. отзывает старые active Authority и завершает active Permit;
3. создаёт successor Context с новыми Genesis digest и ContextID;
4. сохраняет alias, parent, member principal и context kind;
5. создаёт только явно перечисленные initial Authority;
6. ремар-ит normative dependencies на successor IDs;
7. создаёт по одному `MembershipWithdrawalRecord(mode = REDEFINITION)` на старый Context;
8. создаёт `ContextRedefinitionRecord`, содержащий полный proposal, proposal digest, affected set, successor map, withdrawal references и transition reference.

При любой ошибке исходный State возвращается без изменения. Частичное переопределение невозможно.

17.6 Упрощение относительно плебисцита и rc10 draft

Reference profile не содержит:

- `AMENDMENT`;
- `AGREE`, `DISAGREE` и timeout receipts;
- recursive polling/pruning;
- `CUT`;
- `VOLUNTARY_CLOSE`;
- отдельный `REDEFINITION_CONSENT` artifact;
- destructive `PENDING membership withdrawal`;
- alias reservation и cancellation state machine.

Автономия Context сохраняется через exact member authorization каждого затронутого sibling, а атомарность обеспечивается единственным parent-authorized transition. Незатронутые siblings и независимые branches не меняются.

18. Прекращение trust lineage

Если все ключи, представители и заранее установленные recovery mechanisms утрачены, старую trust lineage нельзя безопасно продолжить.

Seed не принимает boolean `all_keys_lost`. Federation должна представить Observation и PASS Verification класса `TRUST_LINEAGE_LOST`, относящуюся к точному child Context.

Принятое termination:

- переводит target subtree в `TERMINATED`;
- прекращает федеративную гарантию;
- отзывает active Authority;
- закрывает активные Permit;
- сохраняет историю;
- не затрагивает независимые branches.

Новое участие требует нового Genesis и нового ContextID с local step 0. Историческая ссылка не создаёт Authority inheritance.

19. Corrections и causal invalidation

Correction является append-only record только для Verification до финального Outcome. Она не удаляет и не переписывает исходный artifact.

Для одного target допускается не более одной effective Correction. Optional `replacement_ref` должен ссылаться на существующую Verification того же Context, Permit, receipt и Observation. `replacement_ref = null` отзывает Verification из effective view. Self-replacement, неоднозначные цепочки и Correction после финального Outcome запрещены.

Outcome не является допустимым target Correction. Это упрощает модель и устраняет расхождение между историческим artifact map и downstream authorization: все вычисления используют один `effective Verification set`, а финальный Outcome остаётся неизменяемым институциональным фактом.

Breach определяется независимо от Outcome polarity. Доказанный первый недопустимый переход инвалидирует его причинных descendants, но не независимые ветви DAG.

Reference rc11 материализует Corrections. Полный автомат breach propagation остаётся normative requirement для последующего расширения и production implementation; rc10 не утверждает завершённую универсальную breach calculus.

20. Whole-state invariants

Reference `validate_state` проверяет минимум:

1. TrustSpaceID, immutable Constitution epoch 0 и Constitution digest.
2. Единственный root Context, отсутствие cycles и неизменяемость root identity.
3. ContextID, вычисленный из parent и Genesis digest.
4. Уникальные aliases только для **ACTIVE** Context и exact live alias index.
5. Current Constitution epoch для active Context.
6. Не более одного active binding на AuthorityKey и отсутствие пересекающихся active scopes.
7. Отсутствие active Authority и active Permit в withdrawn, superseded или terminated Context.
8. Artifact map key = internal artifact ID и существующий owning Context.
9. Exact PermitTerms binding к Decision/readiness и recognized success policy.
10. Verification policy = Permit success predicate = active Constitution rule.
11. Линейную parent-child attenuation и отсутствие двойного attempt budget.
12. Contiguous attempt indices, durable receipts и exact submission index.
13. Observation -> receipt -> Permit binding.
14. Verification -> Observation -> receipt -> Permit binding.
15. Outcome -> полный effective Verification set -> Permit binding.
16. Export commitment, effective Outcome и Import lineage.
17. Verification-only typed Correction до финального Outcome.
18. Transition count, local ordinals, artifact ownership и exact derived causal parents.
19. Universal partition gate для **SUSPENDED** Context.
20. Reconciliation completeness и persistent fork evidence.
21. Уникальность dependency edges, запрет self-reference и существование endpoints.
22. Для каждого **NORMATIVE** edge оба endpoints являются **ACTIVE**.
23. Самостоятельный withdrawal не оставляет active normative dependant.
24. Exact transitive direct-sibling **AffectedSiblingSet**.
25. Canonical proposal digest и exact member authorization set.
26. Atomic old-to-new Context mapping, alias continuity и dependency remapping.
27. Governance records содержат полный proposal и authentication evidence.
28. Exact-context trail для Authority transfer.
29. Context internal root и exact global state root.

21. Machine-readable profile

Пакет содержит:

- 39 JSON Schemas Draft 2020-12;
- one canonical transition discriminated union;
- `initialize_state`, `apply_transition`, `compute_state_root`, `validate_state`;
- 55 trace-based conformance cases;
- positive и negative indexes;
- independent schema/trace/root/mutation audit harness с 367 checks;
- отдельный public-API black-box harness с 25 end-to-end атаками;
- branch guard suite с 252 checks;
- branch-aware coverage reference runtime $733/806 = 90.942928\%$;
- source-bound coverage evidence;
- normative requirements register и traceability matrix.

Каждый conformance case содержит Root Genesis, reachable setup trace, target candidate, exact expected result и optional postconditions. Initial snapshots больше не передаются как произвольные objects.

22. Conformance boundaries

22.1 Подтверждено в rc11 до финального clean-room gate

- schema/corpus/oracle vocabulary alignment и schema enforcement в public reference API;
- достижимость всех 55 setup states;

- materialized state transitions и atomic failure semantics;
- Permit attempt accounting, replay и submission idempotency;
- exact PermitTerms, recognized success predicate и linear attenuation;
- Constitution-resolved Verification policy и complete Outcome aggregation;
- Verification-only Correction и immutable Outcome;
- context-local Authority transfer lineage;
- member-signed standalone withdrawal без active normative dependants;
- canonical atomic sibling redefinition без intermediate pending state;
- exact transitive direct-sibling closure;
- active-only normative dependency graph;
- full proposal/audit evidence retention;
- immutable root Constitution внутри Genesis lineage;
- 55/55 conformance traces;
- 367/367 independent checks;
- 25/25 public black-box attacks;
- 252/252 branch guards;
- branch coverage $733/806 = 90.942928\%$, превышающая обязательный порог 90%.

Числа выше относятся к source-bound working candidate. Финальный статус требует materialized publications, deterministic manifest, archive и повторного clean-room release audit exact bytes.

22.2 Freeze boundary

Статус `DOCUMENTATION_FREEZE_READY` может быть присвоен только после финального clean-room аудита и означает, что normative prose, schemas, reference state machine, conformance corpus, requirements и audit evidence согласованы и готовы к фиксации exact release bytes. Он не повышает reference Python до production implementation.

22.3 Ограничения

- `proof_digest` является входом abstract proof-verifier interface; криптографическая проверка не реализована в reference Python.
- Reference state machine не является production datastore и не доказывает crash durability, multi-process serialization или concurrency safety.
- Distributed consensus не входит в минимальный профиль.
- Изменение root Constitution внутри существующей Genesis lineage не поддерживается; требуется новый Genesis.
- Universal symbolic policy theorem prover и полный breach-propagation model checking не входят в rc10.
- External third-party audit и implementation refinement proof остаются pending.
- Physical-world truth остаётся ответственностью federation procedures и concrete implementations.

23. Центральные инварианты

No recognized Outcome without a valid Context-bound trail:
Decision -> Permit -> ExecutionIntent -> PermitUseReceipt
-> Observation -> Verification -> Outcome

No constitutional state change from self-attested derived predicates.

Destroyed trust lineage is not repaired administratively;
new trust starts from a new Genesis.

A local failure, partition or breach does not globally taint
an otherwise independent subject namespace.

No Verification policy outside the active Constitution.

Historical ancestry is not active membership.

No Context redefinition without one canonical proposal,
exact affected-sibling closure, every member authorization,
and parent REDEFINE_CONTEXT Authority in one atomic transition.

No cross-context Authority transfer without local re-recognition.